

M02 / LEARNER BOOK

SQL foundations

Standalone Zero-to-Job-Ready Data Engineer 2026

14 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE | How to use this module 3

DE02-01 | Database, table, row, column and query mental model 4

DE02-02 | SELECT, aliases and LIMIT 7

DE02-03 | WHERE and comparison operators 10

DE02-04 | AND, OR, NOT and parentheses 13

DE02-05 | ORDER BY, DISTINCT and result inspection 15

DE02-06 | NULL, IS NULL and COALESCE 18

DE02-07 | Arithmetic expressions and safe calculations 21

DE02-08 | Aggregate functions 24

DE02-09 | GROUP BY and grouped grain 27

DE02-10 | HAVING after aggregation 30

DE02-11 | CASE for business categories 32

DE02-12 | String and date basics 35

DE02-13 | Simple subqueries and IN/EXISTS awareness 38

DE02-14 | Debugging and validation workflow 41

START HERE

How to use this module

Open the matching practice folder from the course root: 01_M00_M03/labs. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

This book is written for a beginner. Work through the lessons in order. Read the new words, watch the assigned instructor range, perform the printed example on your laptop, then save an independent attempt before opening the explained review. Ready-made revision is included; you do not need to rewrite the book as notes.

This cumulative package contains M00-M03, G01 and G02. It keeps the completed earlier modules alongside the new SQL modules. The full standalone course continues to M19; the later modules are not delivered as completed books in this batch. There is no fixed time-to-job promise.

Instructor videos open online and use their creators' data. Exact start and stop times are printed with each assigned range. Stop manually at the end: a YouTube start link does not enforce the stop. Published chapter titles, boundaries and runtime were inspected, not complete playback. Read the purpose and any software difference beside the link. A lesson can include written details that its supporting video does not cover; coverage limits remain explicit in WATCH_GUIDE.html.

Study without a Mentor app for now. Keep your own attempts and result evidence using the supplied template. The books contain learner tasks and changed retries; the separate review books contain their answers. The cumulative ZIP includes all required fictional inputs. Never open an answer script before saving your genuine attempt.

DE02-01

Database, table, row, column and query mental model

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

03:16-10:53

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE01-15. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners, 03:16-10:53.](#) [Connect](#) database, SQL and PostgreSQL vocabulary to the book's course.sales table. Follow [SQL_Desk_Setup.pdf](#) before practising, not the old installer later in the video. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

In M01 you opened a workbook and inspected records. A database also stores records, but you ask a database engine to return the ones you need. You do not need to become fast at typing before you can understand that idea. Today you will connect one SQL instruction to one visible result and explain what is stored versus what is displayed. Complete G01 and the SQL Desk Setup guide before running this lesson.

Imagine a shop with many years of records. Opening every record and finding each matching sale by hand becomes awkward. A query expresses a question precisely enough that the database can repeat the work. A data engineer will later put such queries inside dependable processes. We begin with a question small enough to check with our own eyes.

Picture the arrangement

Object Picture it as In this course

Server The program managing the record store

PostgreSQL running on your laptop

Database A separate collection of related objects

standalone_de

Schema A named area inside that database

course

Table Records with a defined set of fields

sales

Query A written request sent to the engine

```
SELECT ... FROM ...;
```

Result The rows returned for that request

The grid below pgAdmin's editor

The filing-cabinet picture is only a mental aid. A schema is not a Windows folder you need to create in File Explorer. In `course.sales`, the dot separates a schema name from a table name. That spelling helps you choose the intended table even when other tables exist.

Our first training table deliberately has one item per order. Its grain is one row per order, and `order_id` identifies that row. Real orders can have several items. M03 will give order headers and item lines separate tables; do not assume this simplified M02 shape is a universal design.

Words before we use them

A column is a named field such as quantity. A row contains the fields for one record. A data type constrains which kind of value a column holds: text for IDs, integer for whole quantities, numeric for prices, date for calendar dates, boolean for true/false, with NULL where a field may be unknown. SQL means Structured Query Language. Execute means send an instruction for the engine to carry out. The SQL editor stores the text you are preparing; the result grid shows a previous execution until you run something again.

Follow with me once

In pgAdmin, open Query Tool on `standalone_de`. Type the following into a new editor tab. Select all three lines and execute them as one statement.

```
SELECT order_id, item, quantity, unit_price
FROM course.sales
ORDER BY order_id;
```

Read it aloud slowly: return the order ID, item, quantity and unit price from the sales table in the course schema, arranged by order ID. SELECT introduces the requested output fields. Commas separate those fields. FROM names the input table. ORDER BY requests a result order; we deepen it in DE02-05. The semicolon marks the end of this statement. Line breaks make the text readable but do not change its meaning here.

The first four results are the records you met in M01:

order_id	item	quantity	unit_price
0007	Notebook	2	49.50
0008	Pen	3	10.00
0009	Folder	1	25.00
0010	Marker	2	30.00

There are four additional records: 0011 Notebook, 0012 Pen, 0013 Ruler and 0014 Bag. The whole result has eight rows and four columns. The table contains other fields, but the query did not request them. Omitting `customer_id` from the display does not erase customer IDs from storage.

What changed, and what did not

This `SELECT` reads data. It does not change a price or create a new sales table. If you close the result tab and execute the same query again against unchanged inputs, the same requested values return. Saving its text as a `.sql` file lets you repeat the request later; saving does not freeze the source table. If a source changes later, rerunning the query can return a different result.

Notice that `order_id` contains 0007, not the number seven used for arithmetic. Its type is text. A price is numeric because multiplication and sums make sense for it. Even if a grid displays both as characters on a screen, their database types and meanings differ. A result grid's formatting is not the definition of a field's type.

If you are stuck

If there is no `course.sales`, use the setup guide's recovery table. If you see only one old result, check which statement was selected and executed. If you wrote a comma after `unit_price`, remove it: `FROM` is a clause, not a fifth selected column. If a column does not exist, compare its exact spelling with the setup file. Do not rename a source column merely to make an accidental spelling work.

Try it yourself

In a blank editor, request `order_id` and `item` from `course.practice_sales`, arranged by `order_id`. Save the SQL and its output in `DE02-01_attempt.sql` and your study record. State the number of rows and columns. Explain whether selecting only two fields deletes the others. You may consult the new-words table, but first try to construct the request yourself.

Changed retry - after your first review

Use `course.retry_sales`. Request `order_id`, `quantity` and `unit_price` arranged by `order_id`. Explain why the order ID and quantity have different roles even though both may contain digit characters.

Revision in your own words

A query is a request; the result is its answer at execution time. `course.sales` names schema and table. `SELECT` chooses displayed fields, `FROM` chooses input, and `ORDER BY` requests their order. SQL text, stored table and displayed result are three different things. Teach it back: point to each of those three objects in your own working session.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-01 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-02

SELECT, aliases and LIMIT

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

01:12:28-01:15:18

[2. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

01:29:35-01:32:43

[3. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

02:09:43-02:12:32

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-01. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[01:12:28-01:15:18. Watch](#) selection. Type our columns and table names in pgAdmin. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[01:29:35-01:32:43. Watch](#) LIMIT. OFFSET and FETCH are optional awareness; practise deterministic LIMIT with the book's ordering. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[02:09:43-02:12:32. Watch](#) naming an output with an alias. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

You can now retrieve named fields. Next you will make the result easier for another person to read and limit a preview to a few records. A preview is useful during inspection, but it is not a total or a representative sample. Complete DE02-01 and keep the loaded tables unchanged.

Picture a printed delivery list with short labels. The label printed on the report need not be the internal name on the warehouse shelf. An alias is that output label. LIMIT caps how many rows the query returns. It does not mean the remaining records were deleted. These operations change the result you request.

Follow with me once

```
SELECT order_id AS sale_id, item AS product
FROM course.sales
ORDER BY order_id
LIMIT 3;
```

The output headers are `sale_id` and `product`. Its rows are 0007 Notebook, 0008 Pen and 0009 Folder. `AS` introduces a temporary name for this output. The source column is still called `item`; another query can still ask for `item`. A meaningful alias tells the reader what a field represents. Use simple lowercase names with underscores so you do not need quoted identifiers at this stage.

The database first has to determine which ordered rows belong at the front of this result. `ORDER BY order_id` supplies that rule. `LIMIT 3` then caps the output at three. Without `ORDER BY`, PostgreSQL may return any three qualifying rows; the table has no promised physical row order for your query to depend on. Running the same un-ordered query twice and seeing the same rows does not turn an accident into a guarantee.

A useful preview versus a misleading conclusion

Suppose the first three products are Notebook, Pen and Folder. You cannot conclude that this shop never sells bags. `LIMIT` did not inspect your question about all products; it deliberately restricted the displayed records. The preview is useful for checking field names and typical shapes. Later, `COUNT` and `DISTINCT` will answer specific whole-dataset questions.

For exploratory work, `SELECT *` requests all available columns. That can help you inspect a small unfamiliar table. For the saved exercises and later transformations, name the fields you actually need. If somebody later adds a column, a named field list makes your intended output clearer. Avoid selecting a column just because it exists.

Small syntax details that prevent confusion

Single quotes create a text value. They do not ask for a column's contents. The next statement returns the literal word `item` on every selected row:

```
SELECT 'item' AS literal_label, item AS actual_product
FROM course.sales
ORDER BY order_id
LIMIT 2;
```

You should see `item/Notebook` and `item/Pen`. Read the first column as “the same label supplied by this query.” Read the second as “the item stored in this record.” Some dialects accept different alias quotation styles; the book consistently uses unquoted simple aliases and single-quoted text values.

If you are stuck

No rows with `LIMIT 0` is an intentional empty preview. Too few rows with `LIMIT 100` can simply mean fewer than 100 exist. An error after `AS` often means the alias contains spaces without valid identifier quoting; change it to a simple name such as `product_name`. If you refer to `product` in a later new query against `course.sales`, remember that `product` was an output alias, not a source column you added.

Try it yourself

Return the first two `course.practice_sales` records ordered by `order_id`. Display `order_id` as reference and item as `product_name`. Save the query and describe exactly what the limit permits you to conclude about the number of records in the whole table.

Changed retry - after your first review

From `course.retry_sales`, return the last two IDs in descending order. Use the output label reference and retain item. `DESC` after `order_id` asks for descending order. Explain whether a second new query must use reference or `order_id` to select the original ID column.

Revision in your own words

`AS` names an output; `LIMIT` caps a result. Use `ORDER BY` whenever which limited rows appear matters. `SELECT item` reads a column, while `SELECT 'item'` returns literal text. A preview is not a row-count check, and an alias is not a permanent table edit.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-02 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-03

WHERE and comparison operators

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

01:21:59-01:29:35

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-02. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners, 01:21:59-01:29:35. Watch](#) filtering and comparison operators. Read the book's quote rules and exact WHERE examples first. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

A business question rarely asks for every record. "Which order lines cost at least 30 per unit?" needs a rule for accepting or rejecting each row. You will express that rule with WHERE, inspect boundary values, and distinguish a unit price from a line amount. Complete DE02-02.

Picture a checkpoint at which each record presents its unit_price. WHERE is the checkpoint rule. A comparison tests two values. A predicate is an expression used as a true/false/unknown test. SQL retains a row in WHERE only when the test is true. The unknown part becomes central in DE02-06.

Follow with me once

```
SELECT order_id, item, unit_price
FROM course.sales
WHERE unit_price >= 30
ORDER BY order_id;
```

Expect 0007 Notebook 49.50, 0010 Marker 30.00, 0011 Notebook 60.00 and 0014 Bag 250.00. The marker passes because >= means "greater than or equal to." With > 30 it would fail. Check that boundary deliberately. A correct-looking list can still be wrong if a requirement said "at least" and the implementation used "more than."

Operator	Meaning	Boundary example
=	Equal	30 = 30 is true
<>	Not equal	30 <> 30 is false
> or <	Strictly above or below	30 > 30 is false
>= or <=	Includes the boundary	30 >= 30 is true

WHERE appears after FROM and before ORDER BY. It does not require you to display the field used in the filter, although displaying it during debugging is helpful. If the task asks about line amount, unit_price alone is the wrong measure: two markers at 30 each have a line amount of 60. We will calculate that explicitly in DE02-07.

Text and date values

```
SELECT order_id, item, ordered_on
FROM course.sales
WHERE item = 'Pen'
ORDER BY order_id;
```

Expect 0008 and 0012. Single quotes tell SQL that Pen is a value. Without them, PostgreSQL looks for an identifier named pen. Our examples use exact stored spelling, including capital P. Do not assume every database uses the same rules for matching case. Text cleaning and case handling come later.

A date condition uses a typed date literal:

```
SELECT order_id, ordered_on
FROM course.sales
WHERE ordered_on >= DATE '2026-09-03'
ORDER BY order_id;
```

IDs 0011 through 0014 match. DATE declares how to interpret the following year-month-day text. That format avoids guessing whether 03/09 means March 9 or September 3. These stored values are calendar dates without times; M03 treats timestamps separately.

Inspect a failure thoughtfully

Zero rows can be a valid answer. First check the exact literal and chosen source. If WHERE item = 'pen' finds none, investigate case rather than deleting the predicate. If you see an old result after a syntax error, the old grid is not the output of the failed query. Read Messages and rerun after correction. Never judge a query by whether it returns some rows; judge whether those are the intended rows.

Try it yourself

From course.practice_sales, show IDs, items and unit prices strictly greater than 30. Predict which row at exactly 30 is excluded before you execute. Save the prediction and actual result.

Changed retry - after your first review

From course.retry_sales, return rows ordered before November 2, 2026, with ID and date. Explain whether a November 2 record belongs in “before November 2.”

Revision in your own words

WHERE accepts individual input rows when its condition is true. Choose both the correct field and the correct operator. Check an exact boundary value, use single quotes for text, and use explicit year-month-day date literals. A successful execution can still answer the wrong business question.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-03 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-04

AND, OR, NOT and parentheses

Watch first

[1. PostgreSQL Tutorial Full Course 2022](#)

55:48-58:12

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-03. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: PostgreSQL Tutorial Full Course 2022, 55:48-58:12. Watch logical operators.](#) The book supplies explicit parentheses, truth reasoning and the NULL caveat. Stop manually at the listed end. Public title, creator, runtime and published chapter boundaries inspected; full playback not verified.

Why this matters, and what you will do

“Paper or Writing products with quantities of at least two” contains two ideas. You will combine them without accidentally letting some rows ignore the quantity rule. Complete DE02-03. This skill matters because a wrong Boolean condition can silently admit the wrong records into a later pipeline.

AND requires both tests to be true. OR accepts either true test. NOT reverses true and false; it does not convert unknown into true. Parentheses mark which smaller question belongs together. Picture two ticket checks: eligible category and sufficient quantity. Each row must pass both overall checks.

Follow with me once

```
SELECT order_id, category, quantity
FROM course.sales
WHERE (category = 'Paper' OR category = 'Writing')
      AND quantity >= 2
ORDER BY order_id;
```

Expected IDs are 0007, 0008, 0010 and 0012. First ask whether each row belongs to either selected category. Then ask whether its quantity reaches two. Folder 0009 and Notebook 0011 belong to Paper but have quantity one, so they fail the overall condition. Ruler 0013 has quantity two but belongs to Tools, so it also fails.

Walk through the trap rather than memorising it

Without parentheses, AND binds more tightly than OR. That means the following query accepts all Paper rows, or Writing rows that meet the quantity rule:

```
SELECT order_id, category, quantity
FROM course.sales
WHERE category = 'Paper'
      OR category = 'Writing' AND quantity >= 2
```

```
ORDER BY order_id;
```

It returns six IDs: 0007, 0008, 0009, 0010, 0011 and 0012. The two quantity-one Paper rows leaked through. Both queries execute; only the first matches our stated question. This is why parentheses are not decoration. Write the grouped idea you intend, even when you could rely on the language's precedence rule.

Row	Eligible category?	Quantity at least 2?	Both?
0007	Yes	Yes	Yes
0009	Yes	No	No
0013	No	Yes	No

Now change the category check to NOT (category = 'Paper'). This means anything except Paper for these non-null category values. It does not mean “find unknown category.” Missing-value logic receives its own lesson next.

Your debugging routine

When a combined condition returns surprising rows, display each field used in the rule. Check one accepted and one rejected row by hand. Then execute each smaller predicate separately if needed. An elaborate condition is easier to understand when you can explain each part. Do not keep swapping AND and OR until the count looks familiar; a count alone may hide a different wrong set of records.

Try it yourself

In course.practice_sales, keep Paper or Bags rows with quantity exactly one. Show ID, category and quantity. Write parentheses around the category condition and explain why P102 should not remain.

Changed retry - after your first review

From course.retry_sales, return records that are not Paper and have quantity at least two. State which two rules each returned row satisfies.

Revision in your own words

Translate the sentence into smaller tests before typing. AND requires both, OR permits either, and NOT reverses a known truth value. Parentheses make the grouping explicit. Validate membership with row IDs, not just an output count. Teach it back by explaining why 0009 leaked through the deliberately wrong guided query.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-04 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-05

ORDER BY, DISTINCT and result inspection

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

01:15:18-01:21:59

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-04. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[01:15:18-01:21:59. Watch](#) ORDER BY and DISTINCT. DISTINCT applies to selected tuples, not to a business claim that two records are the same. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

A useful result must have a clear question, grain and ordering rule. You will sort with ties, list unique combinations, and avoid hiding duplicate business records with DISTINCT. Complete DE02-04. A repeated product name is not necessarily a duplicate sale.

Ascending means lower values first; descending means higher values first. A tie occurs when sort values match. DISTINCT returns unique combinations of the selected output values. It does not inspect your business definition of a duplicate. Picture the difference between a list of every receipt and a catalogue listing each product once.

Follow with me once: deterministic inspection

```
SELECT order_id, item, quantity
FROM course.sales
ORDER BY quantity DESC, order_id;
```

0012 with quantity 4 comes first, then 0008 with 3. The quantity-2 rows are 0007, 0010 and 0013 in ID order, followed by 0009, 0011 and 0014 with quantity 1. The second sort key resolves ties in the first. Without it, the database may put tied rows in any order. A second key is especially useful before a LIMIT when the exact returned records matter.

ORDER BY changes result order, not the order of storage inside the table. A later query still needs its own ORDER BY. DESC applies to the expression immediately before it; ORDER BY quantity DESC, order_id means descending quantity and default ascending ID.

Follow with me once: the selected columns define uniqueness

```
SELECT DISTINCT item
FROM course.sales
ORDER BY item;
```

There are six products: Bag, Folder, Marker, Notebook, Pen and Ruler under the expected simple alphabetical ordering. Your database's collation controls text sorting. Each product name appears once, even though Notebook and Pen each occur on two sales records. The result grain is now one row per distinct item name. No source sale was deleted.

```
SELECT DISTINCT item, unit_price
FROM course.sales
ORDER BY item, unit_price;
```

This returns eight combinations: Notebook appears at 49.50 and 60.00, while Pen appears at 10.00 and 12.50. Adding a selected column changes what DISTINCT compares. If you selected `order_id` as well, every original sale would be distinct because its ID differs.

Why DISTINCT is not a universal repair button

Suppose a join later repeats the same order. Adding DISTINCT can hide repeated displayed values while leaving the underlying modelling mistake unexplained. Worse, two legitimate sales might display the same item and price and be collapsed unintentionally. First state what one input and output row represents, identify the intended key, and investigate where repetition arose. Use DISTINCT when the question genuinely asks for unique selected values.

Inspection you can perform now

Check that the chosen source is correct, list the output fields, inspect the first and last rows under your intended sort, and count the small result by eye. For larger results we will use count queries. If LIMIT is present, label the result a preview. A tidy grid is presentation; it is not proof that filtering, grouping or uniqueness is correct.

Try it yourself

Return unique category names from `course.practice_sales` in ascending category order. Then return its two highest unit prices with their IDs; break ties by ascending ID. Explain why the first result has fewer rows than the source.

Changed retry - after your first review

Return distinct item and unit_price combinations from `course.retry_sales`. Then request the first two records under descending quantity and ascending `order_id`. Explain how the second key resolves the tied quantity of three.

Revision in your own words

ORDER BY belongs to a result request. Add a stable tie-breaker when exact ordering matters. DISTINCT compares the whole selected combination, not just its first column. Unique products and unique sales have different grains; fewer output rows can be correct for one question and wrong for another.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-05 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-06

NULL, IS NULL and COALESCE

Watch first

[1. SQL for Data Analytics - Learn SQL in 4 Hours](#)

01:08:45-01:10:07

[2. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

02:12:32-02:16:15

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-05. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: SQL for Data Analytics - Learn SQL in 4 Hours, 01:08:45-01:10:07.](#)

[Short NULL](#) introduction; the book gives the full missing-value reasoning. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[02:12:32-02:16:15. Watch](#) COALESCE. Substitute a value only when the meaning is justified, not to hide missing source data. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

The shop knows that some orders were paid, knows that others were unpaid, and has not recorded the answer for two orders. You will preserve those three states. Treating an unknown payment as false would create a misleading unpaid list. Complete DE02-05.

A NULL marker represents missing or unknown information, not the text 'NULL', an empty string, or the number zero. Comparisons involving a NULL generally yield unknown. WHERE keeps only true results. IS NULL asks whether the marker is present. COALESCE chooses the first non-null expression from its arguments; it is a fallback rule you choose, not a discovery of the missing fact.

Follow with me once: find what is unknown

```
SELECT order_id, item, is_paid
FROM course.sales
WHERE is_paid IS NULL
ORDER BY order_id;
```

The returned IDs are 0010 Marker and 0013 Ruler. Their displayed Boolean cell may show [null] or a blank depending on pgAdmin preferences. The SQL condition establishes its meaning.

Compare two questions:

```
SELECT order_id
```

```
FROM course.sales
WHERE is_paid = FALSE
ORDER BY order_id;
```

```
SELECT order_id
FROM course.sales
WHERE NOT is_paid
ORDER BY order_id;
```

Both return 0008 and 0012. Neither includes the two unknown records. NOT unknown remains unknown; it does not become true. The complete partition is four paid, two unpaid and two unknown records. We will express and total that partition with CASE later.

The common test that does not work

WHERE is_paid = NULL is not the missing-value test. The equality comparison is unknown for each row, so WHERE keeps none. Even NULL = NULL does not establish equality between two known values. Use IS NULL or IS NOT NULL for these questions. Do not put quotes around NULL unless you truly mean the four-character text, which our Boolean column does not contain.

Follow with me once: a declared fallback

The discount field is missing for 0009. For this particular estimate, suppose the written business rule says, “When the discount has not been supplied, display a provisional rate of zero.” Keep the raw value alongside that estimate:

```
SELECT order_id, discount_pct,
       COALESCE(discount_pct, 0) AS estimated_discount_pct
FROM course.sales
WHERE order_id = '0009';
```

The raw discount remains NULL; estimated_discount_pct is 0. This SELECT did not update storage. The label estimated warns that zero came from an assumption. If the business has supplied no fallback rule, keep the unknown and report the gap. Never silently convert a missing payment status into unpaid merely because COALESCE can do it.

Recovery and validation

If you find zero missing records, inspect the exact predicate, source and current data. If your UI hides null markers, deliberately run IS NULL rather than relying on appearance. If COALESCE reports incompatible types, remember that all candidates must resolve to a suitable common type: a numeric discount and the word Unknown cannot simply be mixed in a numeric calculation.

Try it yourself

In course.practice_sales, separately return IDs with unknown payment and IDs with known unpaid payment. Then display the raw and provisional-zero discount for its one missing-discount row. Explain the assumption behind the provisional value.

Changed retry - after your first review

Use course.retry_sales. Return IDs whose customer_id is missing. Separately return IDs whose discount_pct is missing. Explain why those need not be the same record.

Revision in your own words

Use IS NULL for missingness. Unknown, false, empty text and zero are different states. WHERE retains true, not unknown. COALESCE applies an explicit fallback; preserve the raw value and state any assumption. Teach it back: why does NOT is_paid exclude unknown payments?

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-06 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-07

Arithmetic expressions and safe calculations

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

02:01:55-02:09:43

[2. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

02:16:15-02:20:21

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-06. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[02:01:55-02:09:43. Watch](#) arithmetic and rounding. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[02:16:15-02:20:21. Watch](#) NULLIF and division-by-zero protection. Our book distinguishes numeric division from integer division. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

A unit price and a quantity become a line amount only after you specify the calculation. You will calculate that amount, handle fractional division, and avoid converting an undefined ratio into a believable number. Complete DE02-06. All course currency amounts are fictional INR; the arithmetic does not include tax unless a task explicitly adds it.

An expression combines values and operators to produce a value. A cast requests a data type conversion. A denominator is the divisor in a fraction. NULLIF(a,b) returns NULL when a equals b; otherwise it returns a. We use that last expression to mark a zero-denominator ratio as undefined instead of dividing by zero.

Follow with me once

```
SELECT order_id, quantity, unit_price,  
       quantity * unit_price AS gross_amount  
FROM course.sales  
ORDER BY order_id;
```

The eight gross amounts are 99, 30, 25, 60, 60, 50, 40 and 250. The first calculation is 2 multiplied by 49.50. Aliasing it gross_amount makes the unit and meaning clearer than an automatically generated expression heading. Numeric output may include trailing zeros; 99.00

and 99 are numerically equal.

Multiplication takes place before addition unless parentheses say otherwise. A hypothetical per-order fee of 20 added to a two-unit purchase at 49.50 is $2 * 49.50 + 20 = 119$. The expression $2 * (49.50 + 20)$ gives 139 because it applies the fee to each unit. SQL can execute both perfectly. You must decide which matches the business rule. In M03 we will prevent real order fees being repeated across multiple item lines.

A known discount and an unknown discount

For the provisional-zero rule stated in the previous lesson:

```
SELECT order_id,  
       quantity * unit_price AS gross_amount,  
       quantity * unit_price  
         * (1 - COALESCE(discount_pct, 0)) AS estimated_net  
  
FROM course.sales  
WHERE order_id IN ('0007', '0009')  
ORDER BY order_id;
```

IN here means “equal to either listed ID”; DE02-13 deepens it. 0007 has gross 99 and estimated net 89.10 with its known 10% discount. 0009 has gross and estimated net 25 under the provisional-zero assumption. Omitting COALESCE would leave the second net NULL, honestly reflecting the missing rate. Which version is appropriate depends on the declared purpose; the estimated label matters.

Division: inspect the types

```
SELECT 1 / 2 AS integer_result,  
       1.0 / 2 AS fractional_result,  
       1.0 / NULLIF(0, 0) AS undefined_ratio;
```

PostgreSQL returns 0, 0.5 (possibly more decimal places), and NULL. Dividing two integers truncates toward zero. Introducing a numeric operand keeps the fractional result. NULLIF turns the zero divisor into a null marker, so the ratio is undefined instead of causing a division-by-zero error. A later COALESCE to zero would assert a policy; it would not turn an undefined ratio into a measured zero.

A weighted average later requires total value divided by total units, not the average of prices regardless of quantity. At this stage, practise recognising the numerator, denominator and unit before writing the expression. Use exact numeric types for these classroom currency amounts; rounding a display is separate from choosing the underlying calculation.

If you are stuck

Wrong but plausible totals usually need inspection of the operands and parentheses. Show quantity and unit_price beside the result while debugging. A NULL result may come from a missing operand. A zero fraction may come from integer division. Do not remove safeguards merely to eliminate an error; explain what the safeguarded output means.

Try it yourself

Calculate ID and gross_amount for all course.practice_sales rows. For P102, calculate the net after its known discount without a missing-value fallback. State both amounts and how you checked them by hand.

Changed retry - after your first review

Calculate gross amounts for course.retry_sales. Separately compute a return rate using 3 returned units and 0 sold units, preserving an undefined denominator with NULLIF. Explain why zero percent would misrepresent that case.

Revision in your own words

Name measures with their meaning and unit. Check operator order and fee grain. Numeric types affect division. NULL arithmetic stays unknown; COALESCE needs a declared rule. NULLIF protects a ratio from a zero denominator while preserving its undefined meaning.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-07 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-08

Aggregate functions

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

01:55:40-02:01:55

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-07. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners, 01:55:40-02:01:55. Watch](#) aggregate functions. Our lesson adds COUNT(*) versus COUNT(column), empty inputs and business meaning. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

The shop asks, “How many sales records do we have, and what is their total gross amount?” It needs a summary rather than eight separate lines. You will choose an aggregate according to the question and inspect how missing values affect it. Complete DE02-07.

An aggregate function combines values from several input rows into a summary. COUNT counts, SUM adds, AVG computes an arithmetic mean, and MIN/MAX return the smallest/largest non-null value. With no GROUP BY, the following request treats all qualifying inputs as one group. Picture placing eight receipts into one envelope and writing a few summary facts on its front. The envelope label is not another sale.

Follow with me once

```
SELECT COUNT(*) AS sale_rows,  
       COUNT(is_paid) AS known_payment_rows,  
       SUM(quantity * unit_price) AS gross_total,  
       AVG(quantity * unit_price) AS mean_sale_amount,  
       MIN(unit_price) AS lowest_unit_price,  
       MAX(unit_price) AS highest_unit_price  
FROM course.sales;
```

Expected: 8 sale_rows, 6 known_payment_rows, 614 gross_total, 76.75 mean_sale_amount, 10 lowest_unit_price and 250 highest_unit_price. The one output row represents the whole input selection. It does not represent one of the eight original orders.

Check the total yourself: $99 + 30 + 25 + 60 + 60 + 50 + 40 + 250 = 614$. The mean sale amount is $614 / 8 = 76.75$ because all eight line amounts are known. MIN(unit_price) answers a per-unit price question, while AVG(quantity * unit_price) answers a per-sale amount question. Similar-looking currency columns can describe different measures.

COUNT is two different questions

COUNT(*) counts input rows regardless of which fields are null. COUNT(is_paid) counts only rows where is_paid has a non-null value. Both TRUE and FALSE count because both are known values. It is not a count of paid orders. This distinction is easy to miss if a Boolean field looks like a checkbox on the screen.

To count paid records alone, filter to them first:

```
SELECT COUNT(*) AS paid_rows,  
       SUM(quantity * unit_price) AS paid_gross  
FROM course.sales  
WHERE is_paid = TRUE;
```

Expect 4 and 434. The records are 0007, 0009, 0011 and 0014. WHERE chooses records before this aggregation. Two known-unpaid records total 80, and the two unknown-payment records total 100. Their disjoint totals reconcile: $434 + 80 + 100 = 614$.

Empty input is not the same as a measured zero

```
SELECT COUNT(*) AS rows_found,  
       SUM(quantity * unit_price) AS gross_total  
FROM course.sales  
WHERE item = 'Laptop';
```

The result is one summary row containing 0 and NULL. COUNT reports no records; SUM has no non-null values to add and returns NULL. If a report's agreed convention displays an empty selection as zero, COALESCE(SUM(...), 0) can implement that convention. Keep COUNT alongside it so “nothing observed” is not confused with records whose measured amounts add to zero.

Most ordinary aggregates ignore null input expressions. That does not mean they repair missing source facts. If a rate is missing, calculating AVG over only known rates uses a different denominator from assuming missing rates are zero. The business meaning determines which, if either, is appropriate.

If you are stuck

An error about order_id needing GROUP BY means you mixed an individual record field with a whole-selection summary. For this lesson, remove the record field and label the whole group clearly. If COUNT(is_paid) gives six instead of four, reread which question it answers. If the grid has eight rows, you may have run the earlier row-level query rather than the aggregate.

Try it yourself

For course.practice_sales, return one summary row with source record count, known-payment count, gross total and mean gross amount. Then calculate the count and gross amount of known-unpaid records only. State the grain of each output.

Changed retry - after your first review

Summarise course.retry_sales with total rows, known-payment rows, total quantity, gross total and mean sale amount. Explain why total quantity is not the denominator of the mean sale amount.

Revision in your own words

Choose the aggregate by the question. COUNT(*) counts rows; COUNT(field) counts non-null values, including FALSE. A whole-selection aggregate returns one summary row even for an empty selection. SUM over no input is NULL. Reconcile totals and explain the denominator of every average.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-08 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-09

GROUP BY and grouped grain

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

01:43:10-01:46:41

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-08. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners, 01:43:10-01:46:41. Watch](#) GROUP BY and state the new output grain before comparing the book's grouped totals. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

A grand total is useful, but the shop also wants total amount by product category. You will group records, predict the new grain and reconcile category subtotals to the source total. Complete DE02-08. GROUP BY is the SQL version of the grouping idea you used in a PivotTable, with an explicitly written rule.

Picture sorting receipts into named trays: Bags, Paper, Tools and Writing. Each tray produces one summary row. The number of trays is not the number of receipts. A grouping key defines which tray a record belongs to. Grouped grain tells us what one resulting summary row represents.

Follow with me once

```
SELECT category,  
       COUNT(*) AS sale_rows,  
       SUM(quantity * unit_price) AS gross_total  
FROM course.sales  
GROUP BY category  
ORDER BY category;
```

category	sale_rows	gross_total
Bags	1	250.00
Paper	3	184.00
Tools	1	40.00
Writing	3	140.00

Eight input sale rows become four category summaries. Check both kinds of reconciliation: $1 + 3 + 1 + 3 = 8$ records, and $250 + 184 + 40 + 140 = 614$ currency units. Both checks matter. A result can have the right grand total but mislabel a category, so also inspect at least one group manually. Paper combines Notebook 99, Folder 25 and Notebook 60.

SELECT asks for the grouping key and aggregate values. GROUP BY category says that rows sharing that category feed the same aggregate. ORDER BY category only arranges the resulting summaries. Grouping and sorting are separate operations, even if a small result seems to arrive sorted without an explicit request.

Adding a key changes the question

```
SELECT category, ordered_on,  
       SUM(quantity * unit_price) AS gross_total  
FROM course.sales  
GROUP BY category, ordered_on  
ORDER BY category, ordered_on;
```

The grain is now one row per category per date. There are eight such combinations in this deliberately small dataset. Paper has separate summaries on September 1, 2 and 3. Adding ordered_on did not merely provide a harmless descriptive label; it changed the grouping rule and the output grain. In a larger source there might be many sales in each category/date combination.

Conversely, trying to select an arbitrary item while grouping only by category makes PostgreSQL ask you to resolve ambiguity. Which item should represent Paper: Notebook or Folder? Our learning queries select grouping keys and aggregates explicitly. Do not add all selected columns to GROUP BY automatically just to silence the error, because doing so can change the intended question.

Where missing grouping values go

In PostgreSQL, rows with NULL for a grouping expression are collected into one null group. That does not turn their unknown identities into one real known customer. A summary may legitimately show “unknown customer” as a data-quality bucket, but you cannot infer that all those records belong to the same person.

If you are stuck

First write the sentence “one output row per ...” in the query's comment or study record. Then inspect each selected non-aggregate field: does it belong in that grain? Check both input record count and amount reconciliation. A wrong total is not fixed merely by changing ORDER BY; ordering cannot undo incorrect grouping or filtering.

Try it yourself

Group course.practice_sales by category. Report sale_rows and gross_total, order by category, and reconcile counts and amounts to the ungrouped source. Explain which two records belong to the Paper summary.

Changed retry - after your first review

Group course.retry_sales by ordered_on. Return sale_rows, units and gross_total for each day. Reconcile all three measures to the source.

Revision in your own words

GROUP BY changes grain. State the complete grouping key and use named aggregates. Adding another grouping field changes the question. Check group membership, source counts and subtotal reconciliation. A null grouping bucket does not establish a shared real-world identity.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-09 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-10

HAVING after aggregation

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

01:46:41-01:52:08

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-09. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[01:46:41-01:52:08. Watch](#) HAVING. Keep source-row filtering and group filtering separate. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

The shop now wants only categories whose combined gross sales exceed 150. That rule cannot be applied to each sale independently: the combined category total does not exist until the rows are grouped. You will choose WHERE for input-row rules and HAVING for group rules. Complete DE02-09.

Picture the receipt trays again. WHERE decides which receipts enter the room. GROUP BY puts admitted receipts into trays. HAVING decides which completed tray summaries are shown. This is a logical explanation of the query; PostgreSQL may choose a different physical execution plan while preserving the requested meaning.

Follow with me once

```
SELECT category,  
       SUM(quantity * unit_price) AS gross_total  
FROM course.sales  
GROUP BY category  
HAVING SUM(quantity * unit_price) > 150  
ORDER BY category;
```

Bags 250 and Paper 184 remain. Writing 140 and Tools 40 are excluded. Paper qualifies even though none of its individual sales exceeds 150. The distinction is the lesson: the condition belongs to the group total.

Try expressing the wrong question deliberately:

```
SELECT category,  
       SUM(quantity * unit_price) AS gross_total  
FROM course.sales  
WHERE quantity * unit_price > 150  
GROUP BY category  
ORDER BY category;
```

Only Bags 250 remains because WHERE excluded every smaller sale before grouping. Paper never had a chance to accumulate 184. This second query can be valid for a different question, such as “category revenue from individually large purchases,” but it does not answer our first one.

Use both clauses when the sentence requires both

Suppose the request becomes “Among paid sales, show categories with paid gross above 100.” First filter individual records to known-paid, then group and filter summaries:

```
SELECT category,  
       SUM(quantity * unit_price) AS paid_gross  
FROM course.sales  
WHERE is_paid = TRUE  
GROUP BY category  
HAVING SUM(quantity * unit_price) > 100  
ORDER BY category;
```

Expect Bags 250 and Paper 184. Unknown-payment rows are excluded because the question explicitly says paid. If the question concerned all sales, this filter would be inappropriate. A correct query follows the actual question, not a habitual clause pattern.

The output alias paid_gross is useful in ORDER BY, but PostgreSQL does not generally let you reference that SELECT alias in HAVING. Repeat the aggregate there, or later use a CTE to name an intermediate result. If an instructor's MySQL query uses its aggregate alias in HAVING, use the printed PostgreSQL form above.

Boundary and reconciliation checks

“Above 150” means > 150 . “At least 150” means ≥ 150 . Because HAVING intentionally removes groups, the retained totals need not equal the original grand total. Check the full grouped result before the HAVING filter, then check which groups the condition removes. Reconciliation should respect the stage you are checking.

Try it yourself

Using course.practice_sales, return category totals strictly above 100. Then explain why filtering individual line amounts above 100 before grouping would miss one valid category.

Changed retry - after your first review

From course.retry_sales, keep only groups with at least two sales records per category. Return category, count and gross total. Explain why COUNT(*) belongs in HAVING here.

Revision in your own words

WHERE filters rows before grouping; HAVING filters completed groups. Translate each condition into the stage at which its required value exists. Inspect the unfiltered grouped result before blaming HAVING. PostgreSQL's HAVING should use the aggregate expression here rather than its SELECT alias.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-10 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-11

CASE for business categories

Watch first

[1. Learn SQL Beginner to Advanced in Under 4 Hours](#)

01:26:39-01:35:16

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-10. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn SQL Beginner to Advanced in Under 4 Hours, 01:26:39-01:35:16.](#) [Watch](#) CASE. Alex uses MySQL; our CASE expressions run in PostgreSQL. Use the book's single quotes and own table names. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

A payment flag is stored as true, false or unknown, but a report needs readable group names. You will translate an explicit rule into CASE while preserving unknowns and checking that every record goes into exactly one intended bucket. Complete DE02-10.

A CASE expression tests conditions in order and returns a value from the first matching branch. WHEN states the condition; THEN states its result; ELSE supplies the fallback; END finishes the expression. Picture a receptionist asking one question at a time and writing one classification on a receipt. If the first question already accepts the receipt, later branches are not used to reclassify it.

Follow with me once

```
SELECT order_id,  
       CASE  
         WHEN is_paid = TRUE THEN 'Paid'  
         WHEN is_paid = FALSE THEN 'Unpaid'  
         ELSE 'Unknown'  
       END AS payment_group,  
       quantity * unit_price AS gross_amount  
FROM course.sales  
ORDER BY order_id;
```

IDs 0007, 0009, 0011 and 0014 receive Paid; 0008 and 0012 receive Unpaid; 0010 and 0013 receive Unknown. Unknown does not satisfy either equality condition, so it reaches ELSE. Using ELSE 'Unpaid' after only a paid test would merge missing evidence with known non-payment. That is a data meaning error.

From row labels to a summary

The expression below counts true payment rows. Each qualifying row contributes the number 1; every other row contributes 0:

```

SELECT SUM(CASE WHEN is_paid = TRUE THEN 1 ELSE 0 END)
       AS paid_rows,
       SUM(CASE WHEN is_paid = FALSE THEN 1 ELSE 0 END)
       AS unpaid_rows,
       SUM(CASE WHEN is_paid IS NULL THEN 1 ELSE 0 END)
       AS unknown_rows,
       COUNT(*) AS all_rows
FROM course.sales;

```

Expected 4, 2, 2 and 8. Four plus two plus two equals eight. This is conditional aggregation: each record contributes to an aggregate according to a condition. It makes the partition visible in one row. For currency instead of counts, use `quantity * unit_price` as the contributing value in each matching branch.

Ordered rules can overlap

Suppose “Large” means gross amount at least 100 and “Medium” means at least 50 but less than 100. Test the larger threshold first:

```

SELECT order_id,
       CASE
         WHEN quantity * unit_price >= 100 THEN 'Large'
         WHEN quantity * unit_price >= 50 THEN 'Medium'
         ELSE 'Small'
       END AS size_group
FROM course.sales
ORDER BY order_id;

```

0014 is Large. 0007, 0010, 0011 and 0012 are Medium. 0008, 0009 and 0013 are Small. The second condition can say `>= 50` because amounts `>= 100` have already matched the first branch. Reversing the two WHEN clauses would label 250 Medium. A more verbose non-overlapping rule can also be correct; clarity matters more than brevity.

CASE without ELSE returns NULL when no condition matches. That can be intentional, but here a labelled Unknown or Small bucket is clearer. The branch outputs need compatible types; do not mix a numeric amount with a free-text error message in a column that is meant to remain numeric.

Try it yourself

For `course.practice_sales`, show ID and `payment_group` with all three states. Then calculate paid, unpaid and unknown gross amounts in a single summary query. Reconcile those three figures to the total gross amount.

Changed retry - after your first review

In `course.retry_sales`, define High as gross amount at least 75, Mid as at least 30 but below 75, and Low as below 30. Return ID and group, then identify the exact-boundary records that test your rule.

Revision in your own words

CASE returns one value per row from the first true branch. Preserve missingness with an explicit rule. Test overlapping thresholds in the intended order, include boundary cases, and reconcile mutually exclusive categories. A friendly label is useful only if its rule is correct.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-11 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-12

String and date basics

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

01:37:45-01:43:10

[2. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

02:20:21-02:27:28

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-11. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[01:37:45-01:43:10. Watch](#) text pattern matching. TRIM, case conversion and cleaning policy are explained in our lesson. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[02:20:21-02:27:28. Watch](#) dates, date arithmetic and extracting date parts. Use the book's fixed dates for repeatable results. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

People type product labels differently, and dates need calendar-aware handling. You will normalise simple text in an output without erasing the original, and extract parts of a genuine date. Complete DE02-11. Deeper dirty-text and timestamp cases remain in M03.

A string is text. TRIM removes ordinary surrounding spaces by default; it does not remove every possible invisible character or repair spelling. LOWER converts text to lowercase. EXTRACT retrieves a date/time component. A date literal declares a calendar value; it is not an arbitrary text format.

Follow with me once: keep source and cleaned value

```
SELECT source_id, item_raw,  
       LOWER(TRIM(item_raw)) AS item_clean  
FROM course.raw_labels  
ORDER BY source_id;
```

A contains two leading spaces and one trailing space around pen; it cleans to pen. B is PEN and also becomes pen. C is empty text and remains empty. D is NULL and remains NULL. E becomes note book, with the internal space preserved. F becomes pen blue, still with its two internal spaces. This operation is deliberately limited: it makes surrounding whitespace and letter case consistent, but it does not decide that Note book and Notebook mean the same product.

Keeping `item_raw` next to `item_clean` makes your change inspectable. You can explain why a value changed rather than showing only the final label. The query has not updated the `raw_labels` table. M03 adds a useful conversion of empty cleaned text into an explicit missing marker.

Follow with me once: work with dates as dates

```
SELECT order_id, ordered_on,  
       EXTRACT(YEAR FROM ordered_on) AS year_number,  
       EXTRACT(MONTH FROM ordered_on) AS month_number,  
       ordered_on + 7 AS follow_up_date  
FROM course.sales  
WHERE order_id = '0007';
```

Expect 0007, 2026-09-01, 2026, 9 and 2026-09-08. Adding integer 7 to a PostgreSQL date means seven days. Calendar arithmetic handles month changes; editing the visible day digits yourself would not. YEAR and MONTH are field names in EXTRACT, not quoted source column names.

To select dates within September, use an inclusive beginning and an exclusive next-month boundary:

```
SELECT order_id, ordered_on  
FROM course.sales  
WHERE ordered_on >= DATE '2026-09-01'  
      AND ordered_on < DATE '2026-10-01'  
ORDER BY order_id;
```

All eight rows qualify. The pattern is called a half-open interval: the start is included and the end is excluded. It also adapts well to timestamps, where “the last day” contains many times. In this lesson `ordered_on` has no time component. Do not assume that every field displaying a date-like value has the date type.

Dialect bridge and recovery

Alex's string lesson uses MySQL. LOWER, TRIM and common simple string ideas transfer, but some function names and details do not. Use the PostgreSQL expression printed here. For example, use EXTRACT(MONTH FROM ordered_on), not an assumed MONTH(ordered_on) function. If a date cast fails, inspect the raw format before converting it; do not silently drop inconvenient records.

If surrounding spaces seem to remain, first check whether you are looking at `item_raw` or `item_clean`. If an internal space remains, that is expected for TRIM. If the date result is still an old value, rerun the complete statement and inspect Messages.

Try it yourself

Using `course.practice_sales`, display ID, lowercase item and the date three days after `ordered_on`. Then return its October 1 records using a half-open one-day interval. Explain why the next day is excluded.

Changed retry - after your first review

For R203 in `course.retry_sales`, display the raw item, uppercase item, month number and date seven days later. Explain whether uppercase changes the stored value.

Revision in your own words

Keep raw and cleaned text together. TRIM handles surrounding ordinary spaces, not arbitrary spelling or all whitespace. Date arithmetic uses calendar values; EXTRACT retrieves components. Use explicit typed dates and an inclusive-start/exclusive-end filter. Learn the idea from the video and use the book's PostgreSQL syntax.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-12 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-13

Simple subqueries and IN/EXISTS awareness

Watch first

[1. Learn PostgreSQL Tutorial - Full Course for Beginners](#)

01:32:43-01:35:43

[2. Learn SQL Beginner to Advanced in Under 4 Hours](#)

01:35:16-01:45:58

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-12. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: Learn PostgreSQL Tutorial - Full Course for Beginners,](#)

[01:32:43-01:35:43. Watch](#) IN. Our book expands from a list to a subquery and explains EXISTS. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

[Instructor demonstration: Learn SQL Beginner to Advanced in Under 4 Hours, 01:35:16-01:45:58.](#)

[Watch](#) subqueries. MySQL is the video dialect; use the supplied PostgreSQL queries for practice. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

Some questions need the answer to another question first. “Which sales have amounts above the average sale amount?” needs the average before it can decide which rows qualify. You will write that smaller question inside a larger query and recognise list membership and existence checks. Complete DE02-12.

A subquery is a query nested inside another statement. A scalar subquery supplies one value, such as an average. IN tests membership in a list or a one-column query result. EXISTS asks whether a subquery returns any row; the actual selected value is not what it tests. Picture a shop manager first writing the average on a note, then comparing each sale with that number.

Follow with me once

```
SELECT order_id, quantity * unit_price AS gross_amount
FROM course.sales
WHERE quantity * unit_price > (
    SELECT AVG(quantity * unit_price)
    FROM course.sales
)
ORDER BY order_id;
```

The inner aggregate produces 76.75. The outer query returns 0007 with 99 and 0014 with 250. Run the inner query separately first if the nesting feels unfamiliar. Its closing parenthesis marks the end of the smaller request. Parentheses are not a new table and do not save that result permanently.

Both references to `course.sales` are intentional: the comparison threshold and the candidate rows use the same whole dataset. If you put a paid-only filter inside the subquery but not outside it, you compare all candidates with a paid-only average. That may be valid, but it is a different business rule. Always state the population used to calculate a threshold.

Membership and an existence question

```
SELECT order_id, category
FROM course.sales
WHERE category IN ('Paper', 'Tools')
ORDER BY order_id;
```

This returns 0007, 0009, 0011 and 0013. It is a shorter form of two equality conditions joined with OR. An IN subquery must supply one comparable column for this simple scalar use; SELECT * from a many-column table is not such a list.

```
SELECT EXISTS (
  SELECT 1 FROM course.sales WHERE is_paid IS NULL
) AS has_unknown_payment;
```

The answer is true because at least one matching row exists. SELECT 1 is a conventional way to say “any matching row is sufficient.” It does not count one row or return all matching IDs. M03 will use EXISTS to connect an outer record with matching inner records.

Why NOT IN needs missing-value care

Suppose a list contains C01 and NULL. For a different customer C02, the comparison with C01 is false and the comparison with NULL is unknown. The database cannot prove that C02 is absent from all list members, so NOT IN can yield unknown and remove a row you expected to keep. For now, avoid building exclusion logic from nullable lists. In M03 we will use NOT EXISTS with an explicit matching condition and separately discuss missing outer keys.

If you are stuck

“More than one row returned by a subquery used as an expression” means the outer expression needed one value but received several rows. Inspect the inner query by itself. Do not add LIMIT 1 simply to hide the problem: decide whether you needed an aggregate scalar, membership with IN, or existence. A query returning a Boolean has a different purpose from one returning a list of IDs.

Try it yourself

Find `course.practice_sales` records above that table's own average gross amount. Return ID and amount. Separately ask whether any record in the practice table has a missing `customer_id`.

Changed retry - after your first review

Find `course.retry_sales` records at or above the `retry` table's average gross amount. State the threshold and explain whether `EXISTS` could replace that scalar threshold without changing the question.

Revision in your own words

Run the inner query separately when learning. A scalar subquery supplies one value, `IN` tests membership, and `EXISTS` tests whether any match exists. State the threshold population. Beware nullable `NOT IN` lists; do not disguise a many-row error with an arbitrary `LIMIT`.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-13 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.

DE02-14

Debugging and validation workflow

Watch first

[1. SQL for Data Analytics - Learn SQL in 4 Hours](#)

01:20:15-01:22:35

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: DE02-13. Video: Instructor watch ranges appear below. Read the vocabulary, perform the guided example and save an independent attempt before review.

[Instructor demonstration: SQL for Data Analytics - Learn SQL in 4 Hours, 01:20:15-01:22:35.](#) [Watch logical](#) query order. The book adds a repeatable debugging and evidence checklist. Stop manually at the listed end. Public YouTube title, creator, runtime and expanded publisher chapter list inspected. No full-playback or transcript-completeness claim.

Why this matters, and what you will do

Being able to repair and prove a small query is more useful than copying a longer one you cannot explain. You will diagnose a plausible wrong result, separate syntax from meaning errors and save reproducible evidence. Complete DE02-13. This is an integration lesson; use earlier video segments as targeted refreshers after attempting the diagnosis.

A syntax error means the database cannot parse a statement as written. A semantic error means the statement runs but answers the wrong question. Validation checks that the result matches a stated contract: input, output grain, rules and important totals. A reconciliation compares related counts or amounts expected to agree.

Start with a contract you can read aloud

The request is: “For known-paid sales only, return one row per category, show paid gross amount, and keep categories with paid gross at least 100.” Before opening a query, identify the input as `course.sales`, the row filter as `is_paid = TRUE`, the output grain as `category`, the measure as `quantity times unit price summed`, and the threshold as a group rule. These five choices give you something concrete to test.

A colleague offers this query:

```
SELECT category,
       SUM(unit_price) AS paid_gross
FROM   course.sales
WHERE  is_paid IS NOT NULL
GROUP BY category
HAVING SUM(unit_price) >= 100
ORDER BY category;
```

It executes, but it includes known-unpaid records and sums prices without quantities. On our source it returns Bags 250 and Paper 134.50. The presence of familiar category names does not prove correctness. The word `paid_gross` is just an alias; SQL does not check that the formula deserves the name.

Follow the repair one decision at a time

First inspect source IDs, quantities, prices and payment states. Correct the row rule to TRUE, then calculate each row's amount, then group, then apply the group threshold. The repaired query is:

```
SELECT category,
       SUM(quantity * unit_price) AS paid_gross
FROM course.sales
WHERE is_paid = TRUE
GROUP BY category
HAVING SUM(quantity * unit_price) >= 100
ORDER BY category;
```

Expect Bags 250 and Paper 184. A separate paid-source total is 434. Here both paid categories pass the threshold, so displayed totals also sum to 434. That equality happens because no paid category was filtered out; do not assume a thresholded result must always equal the full paid-source total.

Checks that catch different mistakes

Check What it can catch What it cannot prove alone

Run without syntax errors Misspelled names and malformed statements

Correct business meaning

Inspect selected source IDs

Wrong row membership Correct grouped arithmetic

Recompute one row by hand

Wrong operands or discount rule

Every group is correct

Count rows and distinct keys

Unexpected repetition or grouping

Correct measures and labels

Reconcile totals at the same scope

Lost or multiplied amounts Correct labels for every group

Explain without reading the answer

Gaps in your reasoning Performance on a much larger database

A useful small probe changes a boundary deliberately. In a new SELECT expression you can compare `100 >= 100` with `100 > 100`; there is no need to edit the source to test that operator. For larger later pipelines, temporary tables and transactions will let us test changed inputs safely.

Save evidence that lets you recover

Save the original query before repair, the exact database error or observed wrong result, the corrected query, and one short explanation of each change. You can put SQL comments beginning with `--` above your query. You are documenting an actual attempt, not rewriting theory notes. When asking for help, include the lesson ID, table name, query and exact error text without a password. “It does not work” omits the information needed to diagnose it.

Try it yourself

Request: “For `course.practice_sales`, include sales on October 1 or October 2, produce one row per category with gross amount, and keep category totals at least 30.” Build the query from a blank editor. Save the full grouped output before `HAVING`, the filtered output, and the count and gross total of the selected input records. Explain why their scopes differ.

Changed retry - after your first review

For `course.retry_sales`, report gross amount by payment category with Paid, Unpaid and Unknown all preserved. Reconcile the category row counts and amounts to the source. You may repeat a `CASE` in `GROUP BY`; do not use a `SELECT` alias in `WHERE`. Explain what would go wrong if Unknown were put into Unpaid.

Revision in your own words

Write the contract first: source, eligible rows, output grain, measure and final conditions. Repair syntax before meaning, then validate membership, boundaries and reconciliation at the correct scope. Keep the first attempt and changed retry separate. You are ready for M03 when you can build these M02 queries from a blank editor, explain their results and repair a wrong condition without guessing.

Evidence to keep: your attempt, the observed result and its explanation. Save the first attempt before opening DE02-14 in the Explained Reviews book. After review, solve the changed retry and keep it as a separate attempt.